

Deployment Guide: steveditlea.com

Free and low-cost options for hosting the Next.js site

Overview

The steveditlea.com website is built with Next.js 13.2 and uses server-side features (API routes for the contact form, ISR for Contentful content). This means it can't be deployed as a simple static site — it needs a hosting platform that supports Node.js server rendering. This document covers three deployment options ranked by ease of setup.

Quick Comparison

Platform	Free Tier	Free Tier Limitation	Paid Option	Next.js Support
Netlify	Yes	Cannot deploy from private org-owned repos; easiest with a public repo	Personal \$9/mo; Pro \$19/mo per member	Good (via adapter)
Vercel	Yes	Repo must be under a personal account (not an org); non-commercial use only	Pro \$20/mo per seat	Best (native)
Railway / Render	No	N/A	~\$5–7/mo	Good

Option 1: Netlify (Recommended Easiest Path)

Netlify's free tier allows you to deploy directly from a public GitHub repository. Since the repo doesn't contain any secrets (all sensitive values are stored as environment variables on the hosting platform), making it public is safe. Netlify auto-detects Next.js and handles server-side rendering through its adapter.

Steps

- 1. Make the repo public.** Go to the GitHub repo → Settings → scroll to Danger Zone → Change visibility → Make public. Double-check that there is no `.env` file committed to the repo. The `.gitignore` excludes `.env*`, `local`, but if a bare `.env` file exists, delete it and add `.env` to `.gitignore` before making the repo public.
- 2. Create a Netlify account.** Go to netlify.com and sign up using "Continue with GitHub" (using whichever GitHub account has access to the repo).
- 3. Add a new site.** Click "Add new site" → "Import an existing project" → select GitHub → choose the steveditlea.com repo.

4. **Configure build settings.** Netlify should auto-detect Next.js. Accept the defaults. The build command will be `npm run build` (or `next build`) and the publish directory will be `.next`.
5. **Add environment variables.** Before deploying, go to Site settings → Environment variables and add all five variables: `CONTENTFUL_SPACE`, `CONTENTFUL_TOKEN`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_RECIPIENT`. Also add `NODE_VERSION` set to 18.
6. **Deploy.** Trigger the deploy. Netlify will give you a URL like `random-name.netlify.app`.
7. **Test everything** at the Netlify URL before changing DNS: home page loads content from Contentful, Stories/Books/Photos pages render, contact form sends an email.
8. **Configure the custom domain.** In Netlify, go to Domain management → Add a custom domain → enter `steveditlea.com`. Netlify will give you DNS instructions. Then log into GoDaddy and update the DNS records as described in the DNS Cutover section below.

Note: Netlify's Next.js support works through the `@netlify/plugin-nextjs` adapter, which is automatically installed during the build. For a simple site like this one, it handles ISR and API routes without issue.

Paid option: Netlify Personal and Pro

If you'd rather keep the repo private or need more resources, Netlify offers paid plans. The **Personal** plan is **\$9/month** and includes 1,000 credits/month (vs. 300 on free), priority email support, and smart secret detection. The **Pro** plan is **\$19/month per team member** and adds private organization repo support, shared environment variables, 3+ concurrent builds, 30-day analytics, and 3,000 credits/month. For a low-traffic brochure site like this, the free tier should be more than sufficient, but the Personal plan is a reasonable upgrade if you want the convenience of a private repo without making any changes to repository visibility.

Option 2: Vercel

Vercel is built by the same team that makes Next.js, so it has the best native support. The free Hobby tier works well for a site like this. However, Vercel's free tier requires that the GitHub repo be owned by a personal GitHub account (not a GitHub organization). If the repo is currently under a GitHub organization, you'll need to either transfer it to a personal account or fork it.

Steps

1. **Ensure the repo is under a personal GitHub account.** If it's currently under a GitHub organization (like `SteveDitleaNet`), you have two options: (a) transfer the repo to a personal GitHub account (repo Settings → Danger Zone → Transfer), or (b) fork it to your personal account. Vercel's free tier won't auto-deploy from an org repo.
2. **Create a Vercel account.** Go to `vercel.com` and sign up using "Continue with GitHub." This creates a free Hobby account linked to that GitHub identity.
3. **Import the project.** Click "Add New → Project" and select the repo.
4. **Add environment variables.** Before the first deploy, go to Settings → Environment Variables and add all five variables plus `NODE_VERSION=18`.
5. **Deploy and test.** Vercel will give you a URL like `project-name.vercel.app`. Test all pages and the contact form.

6. **Add the custom domain.** In Vercel, go to Settings → Domains → add `steveditlea.com`. Then update DNS at GoDaddy as described below.

Paid option: Vercel Pro

Vercel's free Hobby tier is restricted to personal, non-commercial use and requires the repo to be under a personal GitHub account. The **Pro** plan is **\$20/month per deploying seat** and removes both restrictions — you can deploy from an organization repo and use it commercially. Pro also includes \$20 in monthly usage credit, 1 TB of bandwidth (vs. 100 GB on Hobby), faster builds with no queue, and unlimited free viewer seats for non-deploying collaborators. For a simple brochure site, the free Hobby tier is likely fine, but if Steve wants to keep the repo in a GitHub organization or prefers official commercial-use compliance, Pro is a straightforward upgrade at a predictable monthly cost.

Other Paid Alternatives

If neither Netlify nor Vercel suits your needs, **Railway** (railway.app) and **Render** (render.com) both offer straightforward Node.js hosting at roughly \$5–7/month for a small site, with no restrictions on repo visibility or ownership. Both auto-deploy from GitHub, support environment variables, and handle custom domains with SSL. Setup is similar to the steps above: connect your GitHub repo, add environment variables, deploy, then configure DNS.

DNS Cutover at GoDaddy

After your new deployment is working and tested at its platform-provided URL, you'll need to update the DNS records at GoDaddy so that `steveditlea.com` points to the new host. You'll need access to Steve's GoDaddy account to do this.

48–72 Hours Before the Switch

1. Log in to GoDaddy → My Products → Domains → `steveditlea.com` → Manage DNS.
2. Find the existing A record (for `@`) and any CNAME record (for `www`). Note their current values.
3. Lower the TTL on both records to 600 seconds (10 minutes). This ensures the old records expire quickly once you make the switch.
4. Wait at least the duration of the original TTL before making the actual change.

Making the Switch

The exact DNS values depend on which platform you chose:

Platform	A Record (@)	CNAME (www)
Vercel	76.76.21.21	<code>cname.vercel-dns.com</code>
Netlify	Use Netlify's DNS instructions (shown in Domain management)	Use Netlify's DNS instructions

Railway / Render	Follow platform-specific instructions	Follow platform-specific instructions
------------------	---------------------------------------	---------------------------------------

After updating DNS, you can check propagation at whatsmydns.net. Most changes take effect within 1–2 hours, though full global propagation can take up to 48 hours. Once confirmed, raise the TTL back to 3600 seconds (1 hour).

Environment Variables Reference

Wherever you deploy, you'll need to configure these environment variables in the hosting platform's dashboard. The actual values are shared separately via a secure link.

Variable	Purpose	Where to Find It
CONTENTFUL_SPACE	Contentful space ID	Contentful → Settings → General
CONTENTFUL_TOKEN	Content Delivery API key	Contentful → Settings → API Keys
SMTP_USER	Gmail address for sending contact form emails	Provided in the secure link
SMTP_PASSWORD	Gmail App Password	Google Account → Security → App Passwords
SMTP_RECIPIENT	Where contact form submissions are sent	Steve's email address
NODE_VERSION	Node.js version (set to 18)	Just set it to 18

A Note on the Email System

The contact form uses Nodemailer to send emails via Gmail SMTP. The SMTP_USER and SMTP_PASSWORD values correspond to a Gmail account with 2-Step Verification enabled and an App Password generated. If you want to use a different Gmail account for sending, you'll need to enable 2-Step Verification on that account and generate a new App Password (Google Account → Security → 2-Step Verification → App Passwords). Then update SMTP_USER and SMTP_PASSWORD in the hosting platform's environment variables.

Gmail's free SMTP limit is 100 emails per day, which is more than enough for a contact form. If you'd prefer to use a dedicated email service like Resend or SendGrid in the future, it would require a small code change in `app/api/email/route.ts` — about 20 lines.

Important: Domain Renewal

While making DNS changes at GoDaddy, confirm that auto-renewal is enabled for `steveditlea.com` with a valid payment method attached. A lapsed domain registration would take the site offline and the domain could be acquired by someone else.